AI Coach SDK for iOS

A full-stack Swift SDK and SwiftUI UI Library for integrating AI coaching into your iOS application.

It contains two modules:

- `AICoachSDK` for headless data, networking, and chat APIs.
- `AICoachUI` for drop-in, fully themable SwiftUI interfaces.

Requirements

- iOS 16.0+
- Swift 5.9+
- Xcode 15+

Installation

Swift Package Manager

Add the following to your `Package.swift`:

```
dependencies: [
    .package(url: "https://github.com/your-org/AICoachSDK.git", from: "1.0.0")
]
```

Or in Xcode: **File → Add Package Dependencies** → paste the repository URL.

Quick Start

1. Configure the SDK

Initialize the SDK at app startup with your API key and backend URL:

```
import AICoachSDK

// In your App init or AppDelegate
try AICoach.configure(
    apiKey: "your-api-key",         // From the admin panel
    baseURL: URL(string: "https://your-backend.com")!,
    userId: "current-user-id"       // Your app's user identifier
)
```

2. Drop-in UI (Recommended)

The easiest way to integrate the coach is to use our pre-built SwiftUI components. They are fully featured, accessible, and handle all network states automatically.

```
import SwiftUI
import AICoachUI

struct MyCoachTab: View {
```

```
    var body: some View {
        // Full screen entry point with routing handled
        AICoachEntryView()
            // Customize colors/fonts to match your brand natively
            .aiCoachTheme(
                AICoachTheme(
                    primaryColor: .purple,
                    backgroundColor: .white
                )
            )
    }
}
```

3. Individual UI Components

If you prefer building your own navigation, you can push individual views:

- `CoachSelectionView()` - Browsing available coaches
- `SessionListView()` - Recent user chats
- `ChatView(coach: Coach)` - Start a new chat
- `ChatView(sessionId: UUID, ...)` - Open an existing chat

4. Headless SDK Usage

If you prefer building your own UI entirely, use the `AICoachSDK` core module directly:

```
// Create a session manually
let session = try await AICoach.shared.createSession(
    coachId: "coach-uuid",
    userState: "Beginner, wants to build muscle"
)

// Send messages
let response = try await AICoach.shared.sendMessage(
    sessionId: session.id,
    content: "What exercises should I do today?"
)
print(response.content ?? "No response")
```

4. Get Chat History

```
let messages = try await AICoach.shared.getMessages(sessionId: session.id)
for message in messages {
    print("\(message.sender ?? "?"): \(message.content ?? "")")
}
```

5. List All Sessions

```
let sessions = try await AICoach.shared.getSessions()
for session in sessions {
```

```
    print("\(session.coachName ?? "Coach") - \(session.messageCount ?? 0) messages")
}
```

6. Load Coaches

```
let coaches = try await AICoach.shared.getCoaches()
let selectedCoach = try await AICoach.shared.getCoach(id: coaches[0].id)
print(selectedCoach.name)
```

Configuration Options

```
let config = AICoachConfiguration(
    apiKey: "your-api-key",
    baseURL: URL(string: "https://your-backend.com")!,
    userId: "user-123",
    platform: "iOS",            // Default: "iOS"
    timeoutInterval: 60,        // Default: 60 seconds
    enableLogging: false        // Default: false
)
try AICoach.configure(config)
```

Updating User ID

When your user logs in or switches accounts:

```
AICoach.shared.updateUserId("new-user-id")
```

Error Handling

The SDK uses `AICoachError` for all error cases:

```
do {
    let response = try await AICoach.shared.sendMessage(
        sessionId: session.id,
        content: "Hello!"
    )
} catch AICoachError.unauthorized {
    print("Invalid API key")
} catch AICoachError.rateLimited {
    print("Too many requests, try again later")
} catch AICoachError.networkError(let error) {
    print("Network issue: \(error.localizedDescription)")
} catch AICoachError.serverError(let code) {
    print("Server error: \(code)")
} catch {
```

```
    print("Unexpected error: \(error)")
}
```

Models

ChatSession

Property	Type	Description
`id`	`UUID`	Unique session identifier
`coachId`	`String?`	The coach's identifier
`coachName`	`String?`	The coach's display name
`lastMessagePreview`	`String?`	Preview of the last message
`messageCount`	`Int?`	Total messages in session
`createdAt`	`Date?`	Session creation timestamp

ChatMessage

Property	Type	Description
`id`	`UUID`	Unique message identifier
`sender`	`String?`	`"user"` or `"assistant"`
`content`	`String?`	Message text
`photoUrl`	`String?`	Attached photo URL
`isFromUser`	`Bool`	Convenience: true if sender is user
`isFromCoach`	`Bool`	Convenience: true if sender is assistant
`createdAt`	`Date?`	Message timestamp

Self-Hosted (Docker) Setup

If your client runs their own backend via Docker, simply point the SDK to their server:

```
try AICoach.configure(
    apiKey: "client-generated-api-key",
    baseURL: URL(string: "https://client-server.com:8080")!,
    userId: "user-id"
)
```

Debug Logging

Enable verbose logging during development:

```
try AICoach.configure(
    apiKey: "key",
    baseURL: url,
    userId: "user",
    enableLogging: true  // Prints redacted request metadata in DEBUG builds
)
```

Payload bodies are always redacted in logs, and release builds never emit verbose request or response payload details even when logging is enabled.

Validation

`AICoach.configure(_:)` now validates the SDK boundary before any request is sent:

- Remote backends must use `https://`
- Plain `http://` is only allowed for local loopback development such as `localhost`
- `apiKey`, `userId`, and `platform` must be non-empty
- `SendMessageRequest.content` must be non-empty and at most 8000 characters
- `CreateSessionRequest.userState` must be at most 4000 characters

UI Theming

The `AICoachUI` package provides an extremely flexible `AICoachTheme` that lets you override virtually every color, as well as the typography used in the interface.

```
let customTheme = AICoachTheme(
    primaryColor: .purple,
    backgroundColor: .black,
    cardBackgroundColor: Color.gray.opacity(0.2),
    textColor: .white,
    coachBubbleBackground: Color.purple.opacity(0.2),
    userBubbleBackground: Color.blue.opacity(0.2),
    font: .custom("Inter", size: 16)
)

AICoachEntryView()
    .aiCoachTheme(customTheme)
```

The Markdown renderer automatically adapts to your custom fonts and colors seamlessly.

Verification

Full package build for iOS Simulator:

```
xcodebuild -scheme AICoachSDK -destination 'generic/platform=iOS Simulator' clean
build
```

License

Proprietary. All rights reserved.